

Regularisation

ECMM422

Fabrizio Costa

Winter 2026

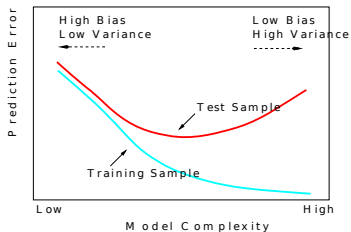
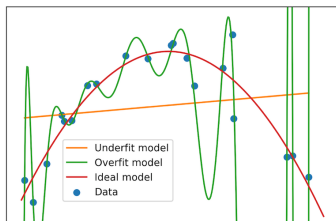


Outline

- 1 Introduction
- 2 Learning Curve
- 3 Regularisation
- 4 Bayesian learning
 - Learning the mean
 - Bayesian perspective on regularisation

Overfitting-Underfitting

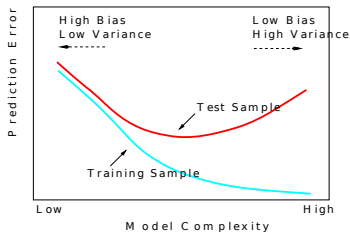
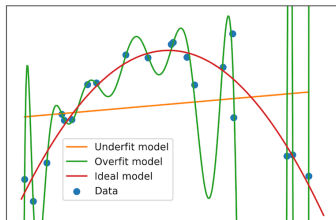
- ▶ The bias-variance analysis shows that a model/hypothesis can have
 - ▶ **high variance**: we say that the model is **overfit**
 - ▶ **high bias**: we say that the model is **underfit**



- ▶ **Q1**: How can we identify which is the case?
- ▶ **Q2**: What can we do to improve the situation?

Overfitting-Underfitting

- ▶ The bias-variance analysis shows that a model/hypothesis can have
 - ▶ **high variance**: we say that the model is **overfit**
 - ▶ **high bias**: we say that the model is **underfit**



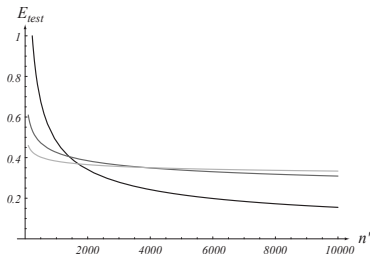
- ▶ **Q1**: How can we identify which is the case?
- ▶ **A1**: analyse CV test error, **learning curves**
- ▶ **Q2**: What can we do to improve the situation?
- ▶ **A2**: train on more data, **regularisation**

Outline

- 1 Introduction
- 2 Learning Curve
- 3 Regularisation
- 4 Bayesian learning
 - Learning the mean
 - Bayesian perspective on regularisation

Learning Curve

- ▶ When choosing between alternative learning algorithms one can make use of their expected prediction error (EPE)
- ▶ However, when datasets are very large the practical training times can be high (days or weeks)
- ▶ **Question:** can we obtain **estimates** of EPE using a smaller data sample?
- ▶ **Answer:** yes, but we cannot use the test error on the smaller sample directly as the estimate: the **asymptotic** behavior can be different from the small sample behavior



Learning Curve

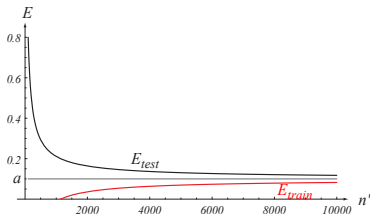
- ▶ For many algorithms and problems the test error has the form:

$$E_{test} = a + \frac{b}{n^\alpha}$$

- ▶ The training error can be represented in a similar way:

$$E_{train} = a - \frac{c}{n^\beta}$$

- ▶ ...where the overfitting phenomenon implies zero error on small datasets
- ▶ ... and the convergence for $n \rightarrow \infty$ to the same error as the test error (i.e. to the Bayes error)



- From the previous expressions we get:

$$E_{train} + E_{test} = 2a + \frac{b}{n^\alpha} - \frac{c}{n^\beta} \quad (1)$$

$$E_{train} - E_{test} = \frac{b}{n^\alpha} + \frac{c}{n^\beta} \quad (2)$$

- If even approximately we assume that $\alpha = \beta$ and $b = c$, then we have simply:

$$E_{train} + E_{test} = 2a \quad (3)$$

$$E_{train} - E_{test} = 2\frac{b}{n^\alpha} \quad (4)$$

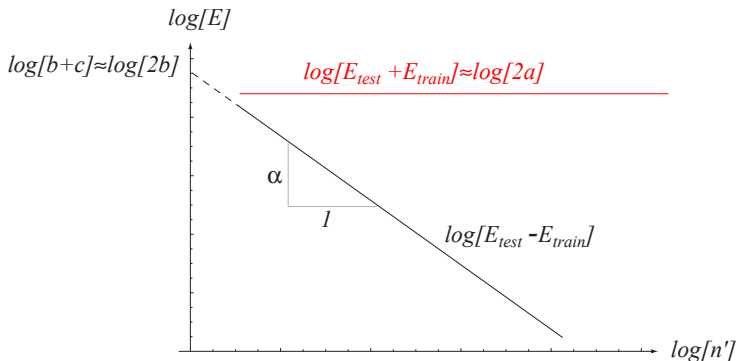
Learning Curve

- If we use a log-log scale, we obtain the linear dependency:

$$\log(E_{train} + E_{test}) = \log(2a) \quad (5)$$

$$\log(E_{train} - E_{test}) = \log(2b) - \alpha \log(n) \quad (6)$$

- $\Rightarrow$ measuring the train and test error, we can estimate α and b



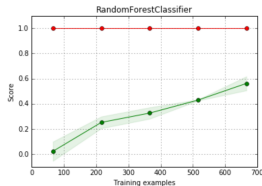
Learning Curve

High capacity + Data

If the model has sufficient capacity/complexity and there is enough data, then the train and test error will converge to the Bayes error

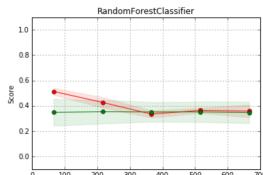
Overfit - Data

If the model is overfit and there is not enough data, the gap between the train and the test error is greater than the Bayes error



Underfit

If the model is underfit then the gap can be small or vanish but the error is greater than the Bayes error



Outline

- 1 Introduction
- 2 Learning Curve
- 3 Regularisation
- 4 Bayesian learning
 - Learning the mean
 - Bayesian perspective on regularisation

William of Occam, 1284-1347 *do not multiply explanations unnecessarily*

Penalise overly-complex solutions
by adding a **regularisation** term to the error.

$$\begin{aligned} E_{\alpha}(w) &= E_{data}(w) + \alpha E_{reg}(w) \\ &= E_{data}(w) + \alpha \int g(\mathcal{F}(x, w)) dx \end{aligned}$$

The function $g()$ allows to quantify a notion of complexity and make it explicitly available to the fitting procedure

The parameter α acts as a "knob" that we can use to regulate the amount of desired complexity

Control curvature

$$E_{reg}(w) = \int \left(\frac{\partial^2 \mathcal{F}(x, w)}{\partial x^2} \right)^2 dx$$

Weight decay regularisation

$$E_{reg}(w) = \sum_i w_i^2$$

Very often small weights correspond to an approximately linear mapping

l_1 regularisation

$$E_{reg}(w) = \sum_i |w_i|$$

Encourages *sparse* mappings with few non-zero weights

Early stopping

Prematurely halt fitting – *hope* that noise is not fitted. Not true regularisation.

Weight decay regularisation

If no training data

$$E = \alpha \sum_i w_i^2 = \alpha |w|^2$$

Gradient descent learning

$$\begin{aligned} w(t+1) &= w(t) - \eta \nabla E \\ &= w(t) - \eta \alpha w(t) \\ &= (1 - \eta \alpha) w(t) \end{aligned}$$

Thus

$$w(t) = (1 - \eta \alpha)^t w(0)$$

As $t \rightarrow \infty$ so $w_i(t) \rightarrow 0$

Weight decay regularisation

True mapping

$$\sin(2\pi x/5) - x/20$$

Noise

Gaussian; variance 0.25

MLP

10 hidden units

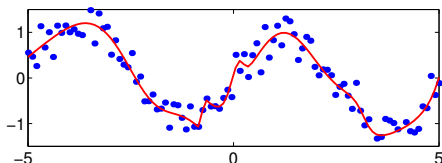
Trained

with 100 examples

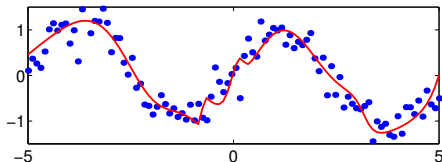
Test

with 100 different examples

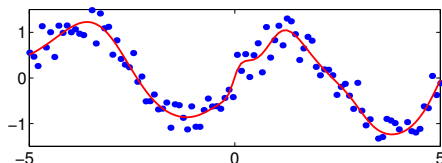
$\alpha = 10^{-5}$
Training, $\alpha = 1e-05$



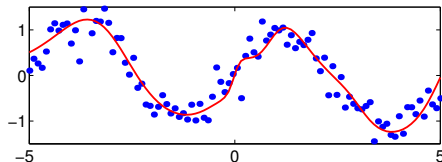
Test, $\alpha = 1e-05$



$\alpha = 10^{-4}$
Training, $\alpha = 0.001$



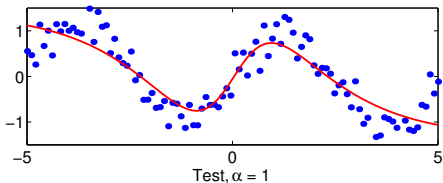
Test, $\alpha = 0.001$



Weight decay regularisation

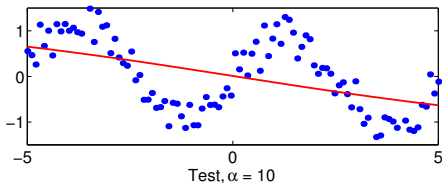
$$\alpha = 1$$

Training, $\alpha = 1$

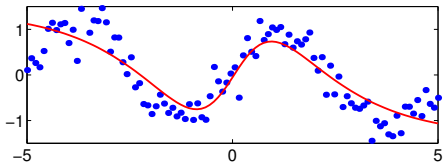


$$\alpha = 10$$

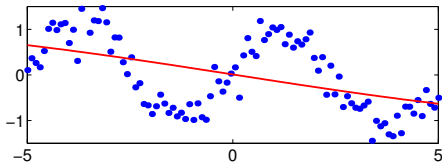
Training, $\alpha = 10$



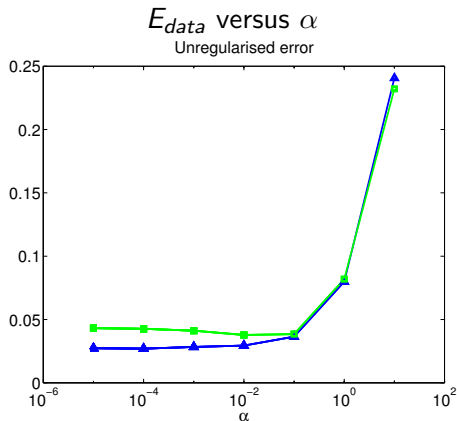
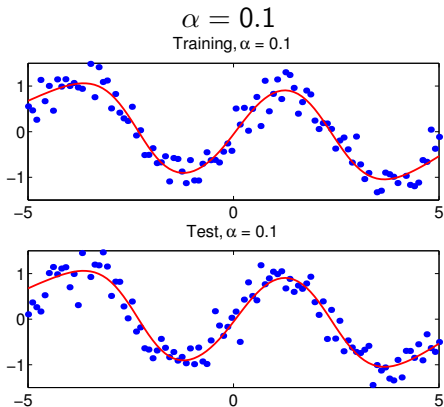
Test, $\alpha = 1$



Test, $\alpha = 10$



Weight decay regularisation



- ▶ Choose a network with “plenty” of hidden units
- ▶ Determine α by cross validation

- ▶ By introducing constraints in the minimisation procedure we reduce the effective space of searchable models (i.e. not all models can be selected)
- ▶ we are effectively trading bias for variance:
 - ▶ stronger constraints imply fewer achievable models
 - ▶ so less freedom to approximate the real function (higher bias)
 - ▶ but also smaller chance to select different models given different samples of the training set (lower variance)

Outline

- 1 Introduction
- 2 Learning Curve
- 3 Regularisation
- 4 Bayesian learning
 - Learning the mean
 - Bayesian perspective on regularisation

Bayesian learning

- 1 Prior beliefs about model parameters encoded in $p(w)$
- 2 Observe data $\mathcal{D}$
- 3 How to update knowledge about parameters having observed $\mathcal{D}$?

Bayesian learning

- 1 Prior beliefs about model parameters encoded in $p(w)$
- 2 Observe data $\mathcal{D}$
- 3 How to update knowledge about parameters having observed $\mathcal{D}$?

Probability of the model given the observed data is

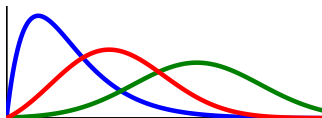
$$p(w | \mathcal{D}) = \frac{p(\mathcal{D} | w)p(w)}{p(\mathcal{D})}$$

$p(w)$	prior probability of w .
$p(\mathcal{D} w)$	likelihood that model with w generated $\mathcal{D}$.
$p(w \mathcal{D})$	posterior probability of w . Probability of w having observed $\mathcal{D}$.
$p(\mathcal{D})$	Normalisation. Can be found from $\int p(\mathcal{D} w)p(w) dw$.

Bayesian learning

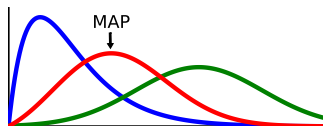
Assumes a prior over model parameters
Finds full posterior **distribution** of parameters, $P(w | \mathcal{D})$

$$p(w | \mathcal{D}) \propto p(\mathcal{D} | w) \times p(w)$$



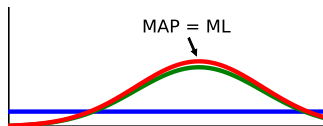
Maximum a posteriori learning

Assumes a prior over model parameters
Chooses the parameters that **maximise** the **posterior**, $P(w | \mathcal{D})$



Maximum likelihood learning

No prior over model parameters
Chooses the parameters that **maximise** the **likelihood** of the data, $p(\mathcal{D} | w)$



Maximum likelihood vs Maximum a posteriori

Maximum likelihood

- ▶ Learn parameters, w_{ML} from training data, $\mathcal{D}$

$$w_{\text{ML}} = \arg \max_w p(\mathcal{D} | w)$$

- ▶ Prediction of new value for input x from

$$y = \mathcal{F}(x, w_{\text{ML}})$$

Maximum a posteriori

- ▶ Learn parameters, w_{MAP} from training data, $\mathcal{D}$

$$w_{\text{MAP}} = \arg \max_w p(w | \mathcal{D})$$

- ▶ Prediction of new value for input x from

$$y = \mathcal{F}(x, w_{\text{MAP}})$$

Incorporates prior knowledge!

Maximum likelihood vs Maximum a posteriori

- ▶ Note that if we maximise the posterior using a non-informed prior (i.e. considering a uniform probability for $p(w)$) then

$$p(w | \mathcal{D}) \propto p(\mathcal{D} | w) \times p(w)$$

$$p(w | \mathcal{D}) \propto p(\mathcal{D} | w) \times \text{const}$$

$$p(w | \mathcal{D}) \propto p(\mathcal{D} | w)$$

- ▶ that is, under a non-informed prior, the maximum a posteriori estimate is the same as the maximum likelihood estimate

Maximum likelihood vs Maximum a posteriori

To understand the importance of being able to specify priors, consider the following example (see [Alex Smola's MAP explanation video](#)):

- ▶ Event $\mathcal{D}$: the student did not present homework
- ▶ Explanations or models w : 1) lazy student; 2) grandma sick; 3) dog ate it; 4) alien abduction;
- ▶ Likelihood $p(\mathcal{D} | w) \approx 1$ i.e. the data can be explained equally well by all models
- ▶ Priors: 1) lazy student=0.8; 2) grandma sick=0.19; 3) dog ate it=0.0099; 4) alien abduction=0.0001;
- ▶ Posterior: $p(w | \mathcal{D}) \propto p(\mathcal{D} | w)p(w)$
- ▶ Maximum likelihood explanation: any of 1,2,3 or 4
- ▶ Maximum a posteriori explanation: lazy student

Bayesian learning of mean

$$p(\mu | \mathcal{D}) \propto p(\mathcal{D} | \mu)p(\mu)$$

Prior for μ

$$p(\mu) = \frac{1}{\sqrt{2\pi}\sigma_0} \exp \left\{ -(\mu - \mu_0)^2 / 2\sigma_0^2 \right\}$$

Represents belief about μ *before* observations are made.

- ▶ σ_0 large: little prior knowledge, vague prior
- ▶ σ_0 small: strong belief about the location of μ

Bayesian learning of mean

$$p(\mu | \mathcal{D}) \propto p(\mathcal{D} | \mu)p(\mu)$$

Prior for μ

$$p(\mu) = \frac{1}{\sqrt{2\pi}\sigma_0} \exp \left\{ -(\mu - \mu_0)^2 / 2\sigma_0^2 \right\}$$

Represents belief about μ *before* observations are made.

- ▶ σ_0 large: little prior knowledge, vague prior
- ▶ σ_0 small: strong belief about the location of μ

Posterior density, having observed data $\mathcal{D} = \{t_n\}_1^N$:

$$p(\mu | \mathcal{D}) \propto \left[\prod_{n=1}^N \frac{1}{\sqrt{2\pi}\sigma} \exp \left\{ -\frac{(t_n - \mu)^2}{2\sigma^2} \right\} \right] \frac{1}{\sqrt{2\pi}\sigma_0} \exp \left\{ -\frac{(\mu - \mu_0)^2}{2\sigma_0^2} \right\}$$

Posterior $\propto$

Likelihood

$\times$

Prior

After tedious algebra ...

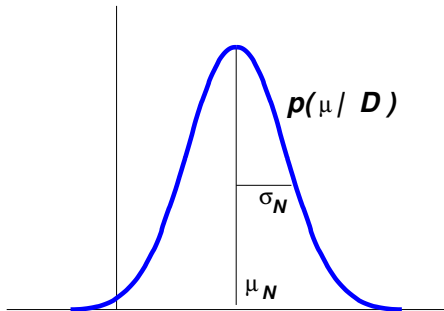
$$p(\mu | \mathcal{D}) = \frac{1}{\sqrt{2\pi}\sigma_N} \exp \left\{ -\frac{(\mu - \mu_N)^2}{2\sigma_N^2} \right\}$$

where

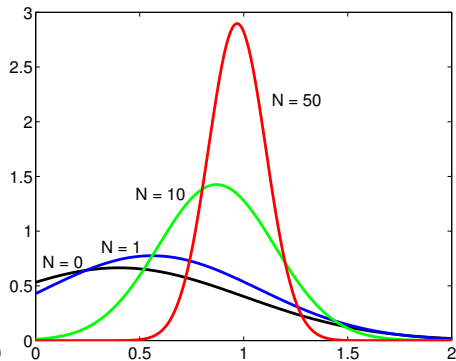
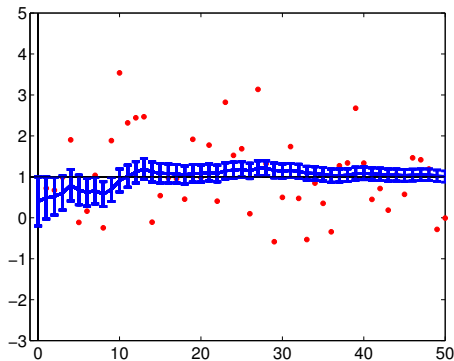
$$\frac{1}{\sigma_N^2} = \frac{1}{\sigma_0^2} + \frac{N}{\sigma^2}$$

$$\mu_N = \sigma_N^2 \left(\frac{\mu_0}{\sigma_0^2} + \frac{N\bar{t}}{\sigma^2} \right)$$

$$\bar{t} = \frac{1}{N} \sum_{n=1}^N t_n$$



Example



Actual: $\mu = 1$
 $\sigma = 1.0$

Prior: $\mu_0 = 0.4$
 $\sigma_0 = 0.8$

- ▶ Bayesian learning gives full posterior distribution
- ▶ MAP estimate of μ is μ_N
- ▶ σ_N measures the *uncertainty* in the estimate of μ_N
- ▶ As $N \rightarrow \infty$, so $\sigma_N^2 \rightarrow \sigma^2/N$
- ▶ As $N \rightarrow \infty$, so $\mu_N \rightarrow \frac{1}{N} \sum_{n=1}^N t_n$

- ▶ Bayesian learning gives full posterior distribution
- ▶ MAP estimate of μ is μ_N
- ▶ σ_N measures the *uncertainty* in the estimate of μ_N
- ▶ As $N \rightarrow \infty$, so $\sigma_N^2 \rightarrow \sigma^2/N$
- ▶ As $N \rightarrow \infty$, so $\mu_N \rightarrow \frac{1}{N} \sum_{n=1}^N t_n$
- ▶ As N becomes large the prior plays a diminishing role and the estimate is determined primarily by the data and the likelihood

The MAP solution can be obtained:

- ▶ Analytically, when the mode(s) of the posterior distribution can be calculated in closed form (e.g. exponential densities, conjugate priors)
- ▶ Via numerical optimisation (this usually requires access to the first or second derivatives)
- ▶ Via an expectation-maximization type of algorithm (no derivatives required)
- ▶ Via a Monte Carlo method using simulated annealing

Bayesian perspective on (weight decay) regularisation

Training amounts to learning the weights, w , from training data $\mathcal{D}$

Maximum likelihood

Maximising log likelihood $\log p(\mathcal{D} | w)$ is equivalent to minimising negative log likelihood or sum squared error:

$$\log p(\mathcal{D} | w) \propto -E_{data}$$

Weight decay regularisation

$$E_{reg} = \alpha \sum_i w_i^2$$
$$-E_{reg} = \log e^{-\alpha \sum_i w_i^2}$$

Bayesian perspective on weight decay regularisation

Bayes rule as logarithms

$$\begin{aligned}\log p(\mathbf{w} | \mathcal{D}) &= \log p(\mathcal{D} | \mathbf{w}) + \log p(\mathbf{w}) - \log p(\mathcal{D}) \\ &= -E_{data} - E_{reg} - \log p(\mathcal{D})\end{aligned}$$

If prior for $\mathbf{w}$ is

$$p(\mathbf{w}) = e^{-\alpha \sum_i w_i^2}$$

- ▶ Minimising $E = E_{data} + E_{reg}$ is equivalent to maximising $p(\mathbf{w} | \mathcal{D})$, the *a posteriori* probability of the weights

Bayesian perspective on weight decay regularisation

$$p(w) = e^{-\alpha \sum_i w_i^2} = e^{-\alpha (w - \mu_0)^2} = e^{-\frac{(w - \mu_0)^2}{\sigma_0^2}}$$

- ▶ where $\mu_0 = 0$ and $\alpha = 1/\sigma_0^2$
- ▶ Prior for w encodes prior belief that weights should be small
- ▶ i.e. the weights are normally distributed with mean 0 and std inversely proportional to α
- ▶ Hyperparameter α controls width of prior:
Small α - large σ wide distribution; weights unconstrained
Large α - small σ narrow distribution; weights constrained (around 0)

- ▶ Predictions in *frequentist* statistics: find an optimum point estimate of the parameters: MLE or MAP, then plug the estimate into the formula for the distribution of data points and find predicted values
- ▶ **Disadvantage:** no account for uncertainty in the parameters: this will underestimate the variance of the predictive distribution
- ▶ **Bayesian approach:** use of the posterior predictive distribution to predict the distribution of a new data point
 - ▶ instead of a fixed point as a prediction, return a distribution over possible points
 - ▶ to do this use the entire posterior distribution of the parameters

- ▶ Prior beliefs and observed data give posterior distribution for parameters

$$p(\mathbf{w} | \mathcal{D}) = \frac{p(\mathcal{D} | \mathbf{w})p(\mathbf{w})}{p(\mathcal{D})}$$

Bayesian prediction

- ▶ Prior beliefs and observed data give posterior distribution for parameters

$$p(w | \mathcal{D}) = \frac{p(\mathcal{D} | w)p(w)}{p(\mathcal{D})}$$

- ▶ Model also gives *predictive likelihood*, $p(y | x, w)$.
Example: $p(y | x, w) \propto \exp -(y - \mathcal{F}(x, w))^2 / 2\sigma^2$

Bayesian prediction

- ▶ Prior beliefs and observed data give posterior distribution for parameters

$$p(w | \mathcal{D}) = \frac{p(\mathcal{D} | w)p(w)}{p(\mathcal{D})}$$

- ▶ Model also gives *predictive likelihood*, $p(y | x, w)$.
Example: $p(y | x, w) \propto \exp -(y - \mathcal{F}(x, w))^2 / 2\sigma^2$
- ▶ Average over all parameters, weighted by the posterior probability

$$p(y | x, \mathcal{D}) = \int p(y | x, w) p(w | \mathcal{D}) dw$$

Bayesian prediction

- ▶ Prior beliefs and observed data give posterior distribution for parameters

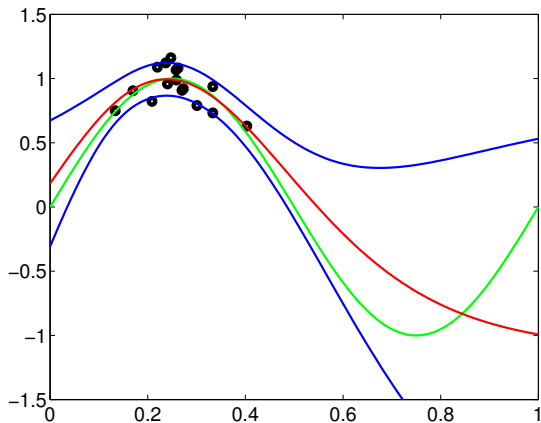
$$p(w | \mathcal{D}) = \frac{p(\mathcal{D} | w)p(w)}{p(\mathcal{D})}$$

- ▶ Model also gives *predictive likelihood*, $p(y | x, w)$.
Example: $p(y | x, w) \propto \exp -(y - \mathcal{F}(x, w))^2 / 2\sigma^2$
- ▶ Average over all parameters, weighted by the posterior probability

$$p(y | x, \mathcal{D}) = \int p(y | x, w) p(w | \mathcal{D}) dw$$

- ▶ Yields a posterior distribution for the prediction
- ▶ Measures uncertainty in prediction
- ▶ Reduces model uncertainty by averaging over a whole class of models

Error bars



Green Function
Red Network
Blue Error bars

- ▶ Over-fitting noise negatively affects generalisation
 - ▶ Cross validation to control over-fitting and poor generalisation
 - ▶ Penalise overly-complex predictors with regularisation term
 - ▶ Weight decay regularisation is very common
-
- ▶ Weight decay may be interpreted as prior belief that parameters should be small
-
- ▶ Bayesian learning can provide posterior distributions of parameters and outputs – error bars